

---

# Mamba on Windows 10 / 11

Install, Shell Enablement, PATH, and YAML Environments

Miniforge3 26.3.2-3 · conda-forge

# Contents

---

|                                                                                         |           |
|-----------------------------------------------------------------------------------------|-----------|
| <b>Placeholders used in this document</b>                                               | <b>4</b>  |
| <b>1. Direct download links</b>                                                         | <b>4</b>  |
| Download + verify from PowerShell                                                       | 5         |
| <b>2. Install</b>                                                                       | <b>5</b>  |
| Option A — interactive                                                                  | 5         |
| Option B — silent / scripted (per-user)                                                 | 5         |
| Option C — all users (run an elevated prompt)                                           | 5         |
| Option D — winget                                                                       | 6         |
| <b>3. Enable <code>mamba</code> in <code>cmd.exe</code> and PowerShell</b>              | <b>6</b>  |
| PowerShell execution policy                                                             | 6         |
| Make <code>mamba activate</code> work in PowerShell 7 and Windows PowerShell separately | 6         |
| Windows Terminal                                                                        | 6         |
| Verify                                                                                  | 7         |
| If <code>mamba activate</code> fails in PowerShell                                      | 7         |
| Permanent fix                                                                           | 7         |
| If it was initialized before                                                            | 7         |
| Check profile permissions                                                               | 7         |
| Immediate workaround                                                                    | 8         |
| If PowerShell startup becomes slow after <code>conda init</code>                        | 8         |
| If <code>mamba</code> / <code>conda</code> commands themselves are slow                 | 8         |
| Undo, if needed                                                                         | 9         |
| <b>4. System / environment variables</b>                                                | <b>9</b>  |
| 4.1 The only PATH entry you should add manually                                         | 9         |
| 4.2 Useful optional variables                                                           | 10        |
| <code>CONDA_*</code> or <code>MAMBA_*</code> ? — you need both                          | 10        |
| <code>envs_dirs</code> is special-cased — and will abort if you set both spellings      | 11        |
| The variables                                                                           | 11        |
| 4.3 <code>.condarc</code> (preferred over env vars for channel config)                  | 13        |
| 4.4 Windows-specific gotchas                                                            | 14        |
| <b>5. Environment YAML: conda packages + pip packages</b>                               | <b>15</b> |

|                                          |           |
|------------------------------------------|-----------|
| 5.1 Example <code>environment.yml</code> | 15        |
| 5.2 Create, update, remove               | 16        |
| 5.3 Ad-hoc installs into an active env   | 16        |
| 5.4 Exporting reproducibly               | 16        |
| 5.5 Practical guidance                   | 17        |
| <b>6. Quick end-to-end check</b>         | <b>17</b> |
| <b>References</b>                        | <b>17</b> |

Mamba is distributed for Windows through **Miniforge3** (conda-forge's minimal installer), which ships both `conda` and `mamba` preconfigured to the `conda-forge` channel. Current release at time of writing: **26.3.2-3** ([conda-forge Miniforge releases](#)).

## Placeholders used in this document

Replace anything in square brackets with your own value. Everything else is a literal Windows path, a real default, or an actual command.

**Table 1. Placeholder conventions**

| Placeholder              | Meaning                                          | Typical value                                                   |
|--------------------------|--------------------------------------------------|-----------------------------------------------------------------|
| [install-root]           | Where Miniforge is installed                     | %USERPROFILE%\miniforge3 (Just Me) or C:\Miniforge3 (All Users) |
| [your-envs-dir]          | Directory holding your environments              | any writable path, e.g. a second drive                          |
| [your-pkgs-dir]          | Package cache directory                          | any writable path on the same volume as your envs               |
| [your-project-dir]       | Root of the project you are working in           | —                                                               |
| [your-local-package-dir] | Path to a package you are installing as editable | —                                                               |
| [your-org] / [your-repo] | GitHub organization and repository               | —                                                               |
| myenv                    | Example environment name                         | pick your own                                                   |

`%USERPROFILE%`, `%APPDATA%`, `%SystemRoot%`, and `$env:USERPROFILE` are genuine Windows variables and should be typed as-is.

## 1. Direct download links

**Table 2. Miniforge download links**

| What                                   | Link                                                                                                                                                                                                                                |
|----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Always-latest Windows x86_64 installer | <a href="https://github.com/conda-forge/miniforge/releases/latest/download/Miniforge3-Windows-x86_64.exe">https://github.com/conda-forge/miniforge/releases/latest/download/Miniforge3-Windows-x86_64.exe</a>                       |
| Pinned 26.3.2-3 installer              | <a href="https://github.com/conda-forge/miniforge/releases/download/26.3.2-3/Miniforge3-26.3.2-3-Windows-x86_64.exe">https://github.com/conda-forge/miniforge/releases/download/26.3.2-3/Miniforge3-26.3.2-3-Windows-x86_64.exe</a> |
| All releases / checksums               | <a href="https://conda-forge.org/miniforge/">https://conda-forge.org/miniforge/</a>                                                                                                                                                 |
| Source repo + README                   | <a href="https://github.com/conda-forge/miniforge">https://github.com/conda-forge/miniforge</a>                                                                                                                                     |

SHA256 for `Miniforge3-26.3.2-3-Windows-x86_64.exe` (78.8 MB), per the [Miniforge releases page](#):

```
14a8635465b5190537ddad6286746ffebbc55a1ed2a7bb14a506595fe3191e1e
```

Requirements: Windows 10 or later, x86\_64. The base environment ships Python 3.14 ([conda-forge/miniforge](#)).

## Download + verify from PowerShell

```
$url = 'https://github.com/conda-forge/miniforge/releases/download/26.3.2-3/Miniforge3-26.3.2-3-Windows-x86_64.exe'
$exe = "$env:USERPROFILE\Downloads\Miniforge3-Windows-x86_64.exe"

$ProgressPreference = 'SilentlyContinue' # <-- REQUIRED in Windows PowerShell 5.1, see note
Invoke-WebRequest -Uri $url -OutFile $exe
$ProgressPreference = 'Continue'

(Get-FileHash $exe -Algorithm SHA256).Hash.ToLower()
# compare against the SHA256 above
```

If **Invoke-WebRequest** **crawls**: in Windows PowerShell 5.1 the progress bar is repainted on every received chunk and dominates runtime — a 79 MB download can take many minutes instead of seconds. Setting `$ProgressPreference = 'SilentlyContinue'` first is typically a 10–50x speedup. PowerShell 7 does not have this problem.

Faster alternatives:

```
# curl.exe ships with Windows 10 1803+ — fastest, shows a real progress meter
curl.exe -L -o "$env:USERPROFILE\Downloads\Miniforge3-Windows-x86_64.exe" `
'https://github.com/conda-forge/miniforge/releases/download/26.3.2-3/Miniforge3-26.3.2-3-Windows-x86_64.exe'

# BITS — resumable, background-friendly, good on flaky links
Start-BitsTransfer -Source $url -Destination $exe
```

## 2. Install

### Option A — interactive

Run the `.exe`. Recommended choices:

- **Just Me (recommended)** — installs to `%USERPROFILE%\miniforge3`, no admin rights needed.
- **Create start menu shortcuts** — leave checked; gives you the "Miniforge Prompt".
- **Add Miniforge3 to my PATH environment variable** — leave *unchecked*. The Miniforge README warns this can cause serious conflicts with other software; Section 4 below shows the safe way ( `condabin` only).
- **Register Miniforge3 as my default Python** — only if no other Python distribution matters to you.
- Install to a path with **no spaces and no non-ASCII characters** (`conda-forge/miniforge`).

### Option B — silent / scripted (per-user)

```
start /wait "" Miniforge3-Windows-x86_64.exe /InstallationType=JustMe /RegisterPython=0 /S /D=%UserProfile%\Miniforge3
```

### Option C — all users (run an elevated prompt)

```
start /wait "" Miniforge3-Windows-x86_64.exe /InstallationType=AllUsers /RegisterPython=0 /S /D=[install-root]
```

`/D=` must be the **last** argument and must be unquoted.

## Option D — winget

```
winget install --id CondaForge.Miniforge3 -e
```

Notes: `/S` = silent, `/InstallationType=JustMe|AllUsers`, `/RegisterPython=0|1`, `/AddToPath=0|1`. Verify the install finished before continuing — `start /wait` blocks until the installer exits, and `%ERRORLEVEL%` is 0 on success.

## 3. Enable `mamba` in `cmd.exe` and PowerShell

With default installer choices, `conda` / `mamba` only work inside the **Miniforge Prompt**. To use them everywhere, run `conda init` once from the Miniforge Prompt (`conda-forge/miniforge`).

Open **Start** → **Miniforge Prompt**, then:

```
conda init cmd.exe powershell
```

`conda init` with no arguments defaults to `cmd.exe` and `powershell` on Windows; `--all` covers every detected shell ([conda init docs](#)).

What this does:

- **cmd.exe** — writes an AutoRun registry value under `HKCU\Software\Microsoft\Command Processor\AutoRun` pointing at `...\miniforge3\condabin\conda_hook.bat`, and puts `condabin` on PATH.
- **PowerShell** — appends a `conda shell.powershell hook` block to your profile (`$PROFILE`, normally `%USERPROFILE%\Documents\PowerShell\Microsoft.PowerShell_profile.ps1` for PowerShell 7, or `...\WindowsPowerShell\...` for Windows PowerShell 5.1).

Close and reopen every terminal afterward.

## PowerShell execution policy

If PowerShell refuses to load the profile (`...cannot be loaded because running scripts is disabled`):

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

## Make `mamba activate` work in PowerShell 7 and Windows PowerShell separately

Each host has its own profile. Run `conda init powershell` from within each host, or check:

```
notepad $PROFILE      # inspect the injected conda block
Test-Path $PROFILE
```

## Windows Terminal

Add a Miniforge profile if you want a dedicated tab (Settings → Add a new profile → command line):

```
%SystemRoot%\System32\cmd.exe /K "%USERPROFILE%\miniforge3\Scripts\activate.bat" "%USERPROFILE%\miniforge3"
```

## Verify

```
mamba --version
conda --version
where.exe mamba      # cmd/PowerShell
mamba info
mamba activate base
```

### If `mamba activate` fails in PowerShell

Symptom: `mamba` runs, but `mamba activate myenv` errors out, or activation only works in the Miniforge Prompt. This means PowerShell was never initialized against the root prefix mamba is actually using. `conda init` (above) covers the conda side; `mamba shell init` is the mamba-side equivalent, and mamba's docs describe `shell init` as the persistent setup while the `shell hook ... | Invoke-Expression` form is only for the current session ([micromamba installation](#)).

### Permanent fix

First find the root prefix actually used by this mamba installation:

```
mamba info
```

Then initialize PowerShell using **that** root prefix. If `mamba info` reports the root prefix as `%USERPROFILE%\local\share\mamba`, run:

```
mamba shell init --shell powershell --root-prefix "$HOME\local\share\mamba"
```

Close all PowerShell and Windows Terminal PowerShell tabs, open a new PowerShell, and test:

```
mamba activate myenv
```

### If it was initialized before

Rebuild the PowerShell initialization block:

```
mamba shell reinit --shell powershell
```

Then close and reopen PowerShell:

```
mamba activate myenv
```

### Check profile permissions

If `shell init` reports an execution-policy issue, inspect your profile and policy:

```
$PROFILE
Get-ExecutionPolicy -List
```

For a normal per-user development setup, permit locally created profile scripts:

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

Restart PowerShell afterward. Do not use an elevated or global policy change unless your organization requires it.

## Immediate workaround

You can run a single command inside the environment without activating it:

```
mamba run -n myenv python --version
mamba run -n myenv python -c "import numpy; print(numpy.__version__)"
```

This is also the right form for scripts, scheduled tasks, and CI. But for interactive work, the shell-hook solution above is the correct fix.

## If PowerShell startup becomes slow after `conda init`

The injected profile block runs `conda shell.powershell hook`, which launches a Python process on *every* new shell — commonly 1–3 s of added startup, worse with antivirus scanning. Options:

```
# 1. See where the time actually goes
Measure-Command { powershell -NoProfile -Command "" } # baseline, profile skipped
Measure-Command { powershell -Command "" }           # with the conda hook
```

- **Cache the hook.** Replace the generated block in `$PROFILE` with a cached version, regenerated only when Miniforge changes:

```
$hookFile = "$env:USERPROFILE\.conda-hook.ps1"
if (-not (Test-Path $hookFile)) {
    & "$env:USERPROFILE\miniforge3\Scripts\conda.exe" shell.powershell hook | Out-File $hookFile
}
. $hookFile
```

Delete `.conda-hook.ps1` after any conda upgrade so it regenerates.

- **Make it lazy.** Drop the hook from the profile and only add `condabin` to PATH ( `conda init --condabin`, or Section 4.1). `mamba install/create/env` all work; you just call `conda activate` via `condabin\conda.bat` or dot-source the hook on demand.
- **Set `auto_activate: false`** in `.condarc` (formerly `auto_activate_base`, still accepted as an alias) so `base` isn't activated in every shell.
- **Exclude `miniforge3`** from Defender real-time scanning (Section 4.4).

## If `mamba / conda` commands themselves are slow

A slow *solve* is a different problem from a slow shell. Check that `channel_priority: strict` is set with `conda-forge` only, that `solver: libmamba` is in `.condarc`, and that Defender exclusions cover `pkgs` and your envs dir. `mamba env create` on a large YAML legitimately takes minutes on first run because it is downloading and extracting hundreds of MB — `mamba env create -f environment.yml -v` shows what stage it is in.



## Undo, if needed

```
conda init --reverse cmd.exe powershell
```

To remove the mamba-side hook as well:

```
mamba shell deinit --shell powershell
```

## 4. System / environment variables

### 4.1 The only PATH entry you should add manually

If you skipped `conda init` (e.g. for CI or a shared build agent), add **just** the `condabin` folder — never the whole `miniforge3` or `Library\bin` (`conda-forge/miniforge`):

```
%USERPROFILE%\miniforge3\condabin
```

GUI route: press `Win`, type "environment variables" → **Edit environment variables for your account** (or **Edit the system environment variables** → *Environment Variables...*) → select `Path` → **Edit** → **New** → paste the path → OK on all dialogs → reopen terminals.

User-scope, from cmd (persists to registry; note `setx` truncates at 1024 chars — prefer the GUI or the PowerShell form below for long PATHs):

```
setx PATH "%PATH%;%USERPROFILE%\miniforge3\condabin"
```

Safer user-scope edit from PowerShell (reads and writes only the *user* PATH, no truncation of the machine PATH):

```
$condabin = "$env:USERPROFILE\miniforge3\condabin"
$old = [Environment]::GetEnvironmentVariable('Path','User')
if ($old -notlike "*$condabin*") {
    [Environment]::SetEnvironmentVariable('Path', ($old.TrimEnd(';') + ';' + $condabin), 'User')
}
```

Machine-scope (elevated PowerShell, for an all-users install at `[install-root]`):

```
$condabin = '[install-root]\condabin'
$old = [Environment]::GetEnvironmentVariable('Path','Machine')
if ($old -notlike "*$condabin*") {
    [Environment]::SetEnvironmentVariable('Path', ($old.TrimEnd(';') + ';' + $condabin), 'Machine')
}
```

## 4.2 Useful optional variables

### CONDA\_\* or MAMBA\_\*? — you need both

**Both prefixes are real, and which one applies depends on the individual setting.** On a Miniforge box you have two different programs reading two different configuration systems:

- **conda** (Python) reads **only** `CONDA_<SETTING>` ([conda `.condarc` docs](#)).
- **mamba** (C++) reads libmamba's configuration. Since mamba 2.x, `mamba` and `micromamba` are "the same code base; only build options vary" ([mamba docs](#)) — the source tree has no separate `mamba/` target. So the `micromamba` rules apply to `mamba` too.

In libmamba, the **default** environment variable for any setting is `MAMBA_` + the uppercased setting name, unless that setting explicitly registers different names (`libmamba/src/api/configuration.cpp`):

```
Configurable&& Configurable::set_env_var_names(const std::vector<std::string>& names)
{
    if (names.empty())
    {
        p_impl->m_env_var_names = { "MAMBA_" + util::to_upper(p_impl->m_name) };
    }
    else
    {
        p_impl->m_env_var_names = names;
    }
}
```

A handful of settings override that default to stay conda-compatible:

**Table 3. Environment variable prefixes, per libmamba setting**

| libmamba setting                 | Env var(s) mamba actually reads                               | Env var conda reads                  |
|----------------------------------|---------------------------------------------------------------|--------------------------------------|
| pkgs_dirs                        | CONDA_PKGS_DIRS <b>only</b> — no MAMBA_PKGS_DIRS exists       | CONDA_PKGS_DIRS                      |
| envs_dirs                        | CONDA_ENVS_DIRS or CONDA_ENVS_PATH (special-cased, see below) | CONDA_ENVS_DIRS / CONDA_ENVS_PATH    |
| channels                         | CONDA_CHANNELS <b>only</b>                                    | CONDA_CHANNELS                       |
| platform                         | CONDA_SUBDIR <b>and</b> MAMBA_PLATFORM                        | CONDA_SUBDIR                         |
| root_prefix                      | MAMBA_ROOT_PREFIX                                             | n/a (conda uses CONDA_ROOT)          |
| channel_priority                 | MAMBA_CHANNEL_PRIORITY                                        | CONDA_CHANNEL_PRIORITY               |
| always_yes                       | MAMBA_ALWAYS_YES                                              | CONDA_ALWAYS_YES                     |
| offline, ssl_verify, no_pin, ... | MAMBA_OFFLINE, MAMBA_SSL_VERIFY, MAMBA_NO_PIN, ...            | CONDA_OFFLINE, CONDA_SSL_VERIFY, ... |

**Practical consequence:** for anything on the `MAMBA_*` default rows, setting only the `CONDA_*` form configures `conda` but *not* `mamba`, and vice versa. If you want the two commands to behave identically, **set both**. This is the part the previous revision of this document got wrong.

`.condarc` avoids the whole problem — libmamba reads `.condarc`, `.mambarc`, `$CONDARC`, and `$MAMBARC`, while conda reads the `.condarc` forms ([mamba configuration](#)). One `.condarc` configures both tools. **Prefer it to environment variables.**

## `envs_dirs` is special-cased — and will abort if you set both spellings

`envs_dirs` deliberately does *not* use the generic mechanism. From the source:

```
// don't use set_env_var_names for CONDA_ENVS_DIRS, since it is a path-sep delimited
// list rather than a YAML list
```

Its hook reads both spellings and **throws a hard error if both are set**:

```
auto conda_envs_path = util::get_env("CONDA_ENVS_PATH");
auto conda_envs_dirs = util::get_env("CONDA_ENVS_DIRS");

if (conda_envs_path && conda_envs_dirs)
{
    const auto message = "The `CONDA_ENVS_DIRS` and `CONDA_ENVS_PATH` environment variables are both set, but only one must be declared. We recommend setting `CONDA_ENVS_DIRS` only. Aborting.";
    throw mamba_error(message, mamba_error_code::incorrect_usage);
}
```

So: **set `CONDA_ENVS_DIRS`, never both**. There is no `MAMBA_ENVS_DIRS`. The list is split on `util::pathsep()`, which is `;` on Windows and `:` elsewhere, so Windows drive letters are safe:

```
constexpr auto pathsep() -> char
{
    if (on_win) { return ';' }
    else      { return ':' }
}
```

Entries are also canonicalized and checked — a path that exists but is not a directory aborts the run, and inaccessible entries are skipped.

## The variables

Values differ depending on whether you did a **Just Me** install (default, `%USERPROFILE%\miniforge3`, no admin) or an **All Users** install ( `[install-root]`, elevated). Set them in the matching scope: `User` for a per-user install, `Machine` for an all-users install.

**Table 4. Environment variables by install scope**

| Setting                                         | Per-user install (User scope) | All-users install (Machine scope)      | Purpose                                               |
|-------------------------------------------------|-------------------------------|----------------------------------------|-------------------------------------------------------|
| CONDA_ENVS_DIRS                                 | [your-envs-dir]               | [install-root]\envs or [your-envs-dir] | Environment locations (both tools; ;-separated)       |
| CONDA_PKGS_DIRS                                 | [your-pkgs-dir]               | [install-root]\pkgs or [your-pkgs-dir] | Package cache (both tools)                            |
| CONDARC                                         | %USERPROFILE%\condarc         | C:\ProgramData\conda\condarc           | Config file (both tools)                              |
| MAMBA_ROOT_PREFIX                               | %USERPROFILE%\miniforge3      | [install-root]                         | Root prefix — <b>mamba</b> and <b>micromamba</b> only |
| CONDA_CHANNEL_PRIORITY + MAMBA_CHANNEL_PRIORITY | strict                        | strict                                 | Must set <b>both</b> to cover conda and mamba         |
| CONDA_AUTO_ACTIVATE                             | false                         | false                                  | Stop auto-activating base — conda-side only           |
| CONDA_DEFAULT_ACTIVATION_ENV                    | myenv                         | myenv                                  | Which env to auto-activate instead of <b>base</b>     |
| PYTHONNOUSERSITE                                | 1                             | 1                                      | Keep %APPDATA%\Python out of env <b>sys.path</b>      |

**Notes:**

- **MAMBA\_ROOT\_PREFIX is legitimate after all** — it is `root_prefix`'s env var in libmamba, and mamba's own error messages tell you to set it when the root cannot be determined. It is *not* needed for a normal Miniforge install (the root is derived from the installation), but if you set it, point it at your Miniforge root so mamba and micromamba share one package cache instead of defaulting to `%USERPROFILE%\micromamba` (**micromamba installation**). `conda` ignores it entirely.
- **auto\_activate\_base was renamed.** The canonical conda setting is now `auto_activate`, with a companion `default_activation_env` (default `base`); `auto_activate_base` remains an alias (**conda configuration reference**).
- **envs\_dirs also influences where package caches land**, and a new named env is created in the first *writable* entry in the list, so order matters.
- **Expansion.** `[Environment]::SetEnvironmentVariable` writes a literal `REG_SZ` — `%USERPROFILE%` will *not* expand later. Either expand it at write time (examples below) or set the value through the GUI, which stores it as `REG_EXPAND_SZ` and does expand it.
- **Machine scope + %USERPROFILE% don't mix.** A machine-scope variable is shared by every account, so never point one at a per-user path. Use machine scope only for genuinely shared locations ( `[install-root]`, `[your-pkgs-dir]`, `C:\ProgramData\conda\condarc` ).
- **Shared package cache.** If several users share `[your-pkgs-dir]` on an all-users box, grant them write access or conda silently falls back to a per-user cache.

Per-user install — set in `User` scope (no admin):

```
[Environment]::SetEnvironmentVariable('CONDA_PKGS_DIRS', '[your-pkgs-dir]', 'User')
[Environment]::SetEnvironmentVariable('CONDA_ENVS_DIRS', '[your-envs-dir]', 'User') # not CONDA_ENVS_PATH too
[Environment]::SetEnvironmentVariable('CONDARC', "$env:USERPROFILE\.condarc", 'User')
[Environment]::SetEnvironmentVariable('MAMBA_ROOT_PREFIX', "$env:USERPROFILE\miniforge3", 'User')
# paired settings: conda and mamba each need their own
[Environment]::SetEnvironmentVariable('CONDA_CHANNEL_PRIORITY', 'strict', 'User')
[Environment]::SetEnvironmentVariable('MAMBA_CHANNEL_PRIORITY', 'strict', 'User')
[Environment]::SetEnvironmentVariable('CONDA_AUTO_ACTIVATE', 'false', 'User')
```

All-users install — set in `Machine` scope (elevated PowerShell):

```
[Environment]::SetEnvironmentVariable('CONDA_PKGS_DIRS', '[install-root]\pkgs', 'Machine')
[Environment]::SetEnvironmentVariable('CONDA_ENVS_DIRS', '[install-root]\envs', 'Machine')
[Environment]::SetEnvironmentVariable('CONDARC', 'C:\ProgramData\conda\condarc', 'Machine')
[Environment]::SetEnvironmentVariable('MAMBA_ROOT_PREFIX', '[install-root]', 'Machine')
[Environment]::SetEnvironmentVariable('CONDA_CHANNEL_PRIORITY', 'strict', 'Machine')
[Environment]::SetEnvironmentVariable('MAMBA_CHANNEL_PRIORITY', 'strict', 'Machine')
[Environment]::SetEnvironmentVariable('CONDA_AUTO_ACTIVATE', 'false', 'Machine')
```

Reopen terminals afterward, then confirm what the shell actually resolved:

```
Get-ChildItem Env: | Where-Object Name -match 'CONDA|MAMBA'
mamba info          # what MAMBA_* resolved to: "base environment", "envs directories", "package cache", "config file"
conda config --show-sources # which values came from env vars vs which .condarc
```

`conda config --show-sources` labels env-var-derived values under an `envvars:` heading, which is the quickest way to confirm a variable is actually being read rather than ignored.

### 4.3 `.condarc` (preferred over env vars for channel config)

Both tools read rc files from several locations, later ones overriding earlier. libmamba additionally accepts a `.mambarc` at each level, which `conda` ignores ([mamba configuration](#)):

**Table 5. rc file locations and precedence**

| Scope        | Path                         | Read by       | Applies to                      |
|--------------|------------------------------|---------------|---------------------------------|
| System       | C:\ProgramData\conda\condarc | conda + mamba | All users (needs admin to edit) |
| System       | C:\ProgramData\conda\mambarc | mamba only    | All users                       |
| Install root | <root prefix>\.condarc       | conda + mamba | Anyone using that installation  |
| User         | %USERPROFILE%\condarc        | conda + mamba | Just you (the usual choice)     |
| User         | %USERPROFILE%\mambarc        | mamba only    | Just you                        |
| Environment  | <env prefix>\.condarc        | conda + mamba | That one environment            |
| Explicit     | \$CONDARC / \$MAMBARC        | conda / mamba | Whatever you point them at      |

Use `.condarc` and skip `.mambarc` entirely — it is the only file both programs read, so one file keeps `conda` and `mamba` in agreement. This is the cleanest way around the split env-var namespaces described in 4.2.

For a **per-user** install, put this in `%USERPROFILE%\condarc`:

```
channels:
  - conda-forge
channel_priority: strict
solver: libmamba
auto_activate: false    # formerly auto_activate_base, still accepted as an alias
envs_dirs:
  - [your-envs-dir]
pkgs_dirs:
  - [your-pkgs-dir]
```

For an **all-users** install, put the shared parts in `C:\ProgramData\conda\condarc` (elevated) and leave per-user overrides in each `%USERPROFILE%\condarc`:

```
# C:\ProgramData\conda\condarc — machine-wide defaults
channels:
  - conda-forge
channel_priority: strict
solver: libmamba
pkgs_dirs:
  - [install-root]\pkgs
```

Write entries without hand-editing, and see which file conda is using:

```
conda config --show-sources          # every .condarc in play, in precedence order
conda config --set channel_priority strict  # writes user .condarc
conda config --system --set channel_priority strict  # writes system .condarc (elevated)
```

## 4.4 Windows-specific gotchas

- **Long paths.** Deep env prefixes break some packages. Enable long paths:

```
# elevated
New-ItemProperty -Path 'HKLM:\SYSTEM\CurrentControlSet\Control\FileSystem' `
  -Name LongPathsEnabled -Value 1 -PropertyType DWORD -Force
```

- **Antivirus / Defender.** Exclude `miniforge3\pkgs` and your envs dir to avoid very slow solves and extraction.
- **Symlink/hardlink warnings.** Keep envs and the package cache on the *same* volume, otherwise every install copies instead of hardlinking.

## 5. Environment YAML: conda packages + pip packages

### 5.1 Example `environment.yml`

```
name: myenv
channels:
  - conda-forge      # keep conda-forge first; do not mix in defaults
dependencies:
  # --- interpreter (pin it explicitly) ---
  - python=3.12

  # --- conda-forge packages ---
  - numpy>=1.26
  - scipy
  - pandas
  - matplotlib
  - h5py
  - scikit-learn
  - jupyterlab
  - pyyaml
  - pytest

  # --- native/binary deps that must come from conda, not pip ---
  - hdf5
  - fftw
  - libblas=*mkl      # build-string pin example

  # --- pip must be listed as a conda dependency before the pip block ---
  - pip
  - pip:
    # anything not on conda-forge, or where you need the PyPI build
    - some-pypi-only-pkg
    - example-pkg==1.2.3
    - another-pypi-pkg
    - "windows-only-pkg; platform_system=='Windows'"
    - r requirements-extra.txt
    - --extra-index-url https://download.pytorch.org/whl/cu124
    - torch==2.4.1+cu124
    - git+https://github.com/[your-org]/[your-repo].git@v1.2.3
    - e [your-local-package-dir]
variables:
  MPLBACKEND: QtAgg
  MY_PROJECT_ROOT: [your-project-dir]
```

#### Key rules:

- The `pip:` mapping lives **inside** `dependencies:` and `pip` itself must be listed as a conda dependency, or conda/mamba will warn and may use a foreign pip.
- Conda resolves and installs everything above the `pip:` block **first**, then hands the pip list to `pip install` in one call. Pip has no visibility into the conda solve, so put every compiled library (HDF5, FFTW, BLAS, GDAL, Qt, CUDA runtime...) on the conda side.
- Pip lines accept normal `pip` syntax: version pins, PEP 508 markers, `-r`, `-e`, `--index-url`, `--extra-index-url`, VCS URLs, local paths.
- `variables:` sets environment variables that apply when the env is activated.

## 5.2 Create, update, remove

```

:: create from file
mamba env create -f environment.yml

:: create under a specific name or prefix, overriding the YAML
mamba env create -f environment.yml -n myenv2
mamba env create -f environment.yml -p [your-envs-dir]\myenv

:: activate
mamba activate myenv

:: apply YAML changes to an existing env; --prune removes what's no longer listed
mamba env update -f environment.yml --prune

:: inspect
mamba env list
mamba list -n myenv

:: delete
mamba env remove -n myenv

```

`mamba` also accepts a spec file directly with `mamba create -f env.yml` ([mamba user guide](#)), and multiple files can be layered by repeating `-f`.

## 5.3 Ad-hoc installs into an active env

```

mamba install -c conda-forge requests rich
python -m pip install --no-build-isolation some-pypi-only-pkg

```

Always call pip as `python -m pip` while the env is active so you cannot hit a pip from another installation.

## 5.4 Exporting reproducibly

```

:: portable across OSes: what you asked for, not every transitive build
mamba env export --from-history > environment.yml

:: full solve including pip packages, same-OS reproducibility
mamba env export > environment.full.yml

:: exact URLs + hashes, single platform, strongest reproducibility
mamba list --explicit > win64-lock.txt
mamba create -n myenv-clone --file win64-lock.txt

```

For multi-platform locking that also covers pip, use `conda-lock`:

```

mamba install -c conda-forge conda-lock
conda-lock lock -f environment.yml -p win-64 -p linux-64
conda-lock install -n myenv conda-lock.yml

```

`mamba create -n my-environment -f conda-lock.yml` also consumes lock files, which must end in `-lock.yml` or `-lock.yaml` ([mamba user guide](#)).



## 5.5 Practical guidance

- Pin `python=` explicitly. Without it, an `env update` can silently move you to a new minor version and rebuild everything.
- Use `channel_priority: strict` with `conda-forge` only. Mixing `defaults` and `conda-forge` is the most common source of broken solves.
- Prefer conda packages when both exist; drop to pip only for PyPI-only projects, private wheels, editable local source, or CUDA wheels from a vendor index.
- Re-running `mamba env update` will *not* uninstall pip packages you removed from the YAML unless you pass `--prune`, and even then pip-installed packages are handled imperfectly. For a clean slate, delete and recreate the env.
- Commit both `environment.yml` (human intent) and a lock file (machine truth).

## 6. Quick end-to-end check

```
mamba --version
mamba env create -f environment.yml
mamba activate myenv
python -c "import numpy, sys; print(sys.executable); print(numpy.__version__)"
python -m pip list --format=columns
```

`sys.executable` should point inside your env prefix (e.g. `%USERPROFILE%\miniforge3\envs\myenv\python.exe`). If it doesn't, your PATH still has another Python ahead of conda — recheck Section 4.1.

## References

- Miniforge repository and Windows install notes — <https://github.com/conda-forge/miniforge>
- Miniforge release list, versions, and SHA256 checksums — <https://conda-forge.org/miniforge/>
- `conda init` command reference — <https://docs.conda.io/projects/conda/en/latest/commands/init.html>
- Mamba user guide (YAML spec files, lock files) — [https://mamba.readthedocs.io/en/latest/user\\_guide/mamba.html](https://mamba.readthedocs.io/en/latest/user_guide/mamba.html)
- Mamba installation docs — <https://mamba.readthedocs.io/en/latest/installation/mamba-installation.html>
- Micromamba installation (root prefix, `shell init`, shell hook) — <https://mamba.readthedocs.io/en/latest/installation/micromamba-installation.html>
- Mamba configuration (rc file search order, `MAMBA_*` env vars) — [https://mamba.readthedocs.io/en/latest/user\\_guide/configuration.html](https://mamba.readthedocs.io/en/latest/user_guide/configuration.html)
- libmamba configuration source — the authority on which prefix each setting reads — <https://github.com/mamba-org/mamba/blob/main/libmamba/src/api/configuration.cpp>
- conda configuration overview — <https://docs.conda.io/projects/conda/en/latest/configuration.html>
- conda `.condarc` usage — <https://docs.conda.io/projects/conda/en/latest/user-guide/configuration/use-condarc.html>
- conda settings reference (`auto_activate`, `default_activation_env`) — <https://docs.conda.io/projects/conda/en/latest/user-guide/configuration/settings.html>